What replay tests miss: verifying a clean-room rewrite of an LLM agent harness

Full method and results (companion to the ARC Prize 2026 paper-track writeup). Every number is tied to a source line in the authors' verification records.

3. Showing the rewrite is equivalent

3.1 Deterministic checks

Live runs of an LLM harness are noisy. An earlier comparison of one configuration across two platforms gave p = 0.86, so a live test alone detects only large changes. We therefore compared the two harnesses first where the model's output can be held fixed, and kept a live test for what remains.

D1, engine replay. The recorded action histories of all 30 game runs of the original's reference run (enf10) were fed through the rewrite's engine adapter. Per-run levels, per-level actions and score had to match exactly. D1 passed in the implementer's test suite; the verifier did not re-run it.

D3, sandbox corpus. We extracted all 1,473 Python cells from the 30 enf10 transcripts. Each harness built a cell's input state with its own code from the same recorded state. A first pass ran the original and recorded the exact sandbox inputs, including every gate backtest source; a second pass fed those inputs to the rewrite. The pass rule was identical stdout, result, error type and action-request stream. To separate real differences from noise, we also ran the original against itself: two runs of the original disagree in 15 cells, from set-iteration order. In the final state, all 1,473 action-request streams and all 637 gate backtests were identical, and the remaining differences were set order and error wording in 13 cells. Crafted probes covered behavior the corpus never exercises, such as class definitions and `except KeyError`.

D2, record/replay. A mock server returned the original's recorded assistant messages in order, per game run, ignoring the request. Both harnesses ran against it on the same 6 games. With the model's output fixed, any difference in what a harness does with it is a harness difference. The pass rule had four parts: identical engine actions, identical gate event logs, identical compact tool results (wording fields excepted), and equal eviction points, meaning the same message count in every request. Two wall-clock features were disabled on both sides to make replay deterministic: the 60 s turn yield and the per-game cap. Usage counts were stand-ins. The original's replays were deterministic across rounds 1, 4, 7 and 9.

D4, competition path. Both harnesses ran against a local gateway simulator in a separate process, as the Kaggle gateway is, with a stub model taking one action per turn. The rule was one scorecard opened and closed, and each game id made once. Both passed with 6 games and with 110 cloned ids. The rewrite's added soft deadline was tested by shifting the notebook start so the deadline fell about 55 s into a run: one scorecard closed with all 903 actions.

8-gram prompt overlap. As a mechanical check of the clean-room rule, we tokenized the string literals of both code bases into lowercase words and searched for shared runs of 8 words. The rule was

no shared run with the third-party prompts. The final state had 0 hits in prompt text; the 11 remaining hits were interface identifiers, such as environment variable names and JSON keys, that the spec requires to match.

Ten rounds to parity. The first round found real differences. Engine actions matched in 20 of 30 runs. The rewrite's prompts were about 2.6 times shorter: its system prompt was 5,113 characters against 14,065. Because context trimming depends on prompt size, eviction points differed in 550 of 983 requests. Other findings included a gate hook called at the wrong time, an action the original rejects, and a different gate count for failed calls. After round 2, actions, gate logs and token attribution matched in 30 of 30 runs. Eviction was the long tail:

Round	Rewrite commit	Requests with a different message count
1	8efa465	550 / 983
2	8fdafee	395 / 1,504
3	5ac22f7	193 / 1,504
4	77a9616	188 / 1,504
5	11edf60	6 / 1,504
6	f62e454	4 / 1,504
8	3a02e90	1 / 1,504
9	254eb94	1 / 1,504
10	c18d184	0 / 1,504

Two things drove the tail. First, when a long turn overflows, the original sends a request containing only the system message. It did this 179 times in 19 of 30 runs, and the rewrite had to reproduce it. Second, matching raw prompt length was not enough. The trimming estimator counts JSON-escaped characters, so the serialized size of every message template had to match. In round 9, one request sat 3 characters over the budget on the original (95,235 against 95,232). The rewrite was about 4 characters smaller there, and whether it crossed the line depended on wall-clock digits printed by model code; it matched the original in 1 of 4 extra replays. Round 10 set the serialized size difference of every template to 0 across 362 paired messages, with the system message (14,065 characters) and tools JSON (1,144) matching exactly. That request then matched in 4 of 4 replays, and eviction matched in 1,504 of 1,504 requests.

Final deterministic result. At rewrite c18d184, D1, D2, D3, D4 and the 8-gram check all passed. D2 matched in 30 of 30 runs on actions, gate logs, token attribution and solver notes, on 1,504 of 1,504 eviction points, and on 362 of 362 previous-turn summary facts.

What this layer cannot test. D2 replays fixed model output. It verifies every model input except prompt text: tool results, image pixels, request parameters, history and eviction, re-sent reasoning, and per-template sizes. It says nothing about how the model reacts to different wording. The spec named this as the highest risk and assigned it to the live test.

3.2 Live non-inferiority test (S1)

Pre-registration. The S1 plan was written and approved before any S1 data existed. The new run was the rewrite on the 6 probe games, 5 passes each (30 game runs), with the original's gate settings. The comparator was the original's enf10 run on Kaggle with the same settings; the original's held-out run used different hardware and is reported for context only. A second run was allowed only if the first was invalid, and a valid run would never be repeated to get a better result.

The original's figures were a mean of 0.500 levels per game run (SD 0.572, n = 30), 18 of 30 runs with at least one accepted world model, and a median generation throughput of 165.85 tok/s. All three pass rules had to hold:

1. Levels, non-inferiority: a one-sided Welch test of H0: mean(rewrite) − mean(original) ≤ −0.5, rejected at α = 0.10.
2. Gate acceptance: 13 to 23 runs of 30 (18 ± 2 binomial SD).
3. Throughput: median within 10% of the original, 149.3 to 182.4 tok/s.

Rules checked against noise before data. The first draft used a 0.3 margin and a 16–20 band. We computed how often these rules would fail a rewrite that plays exactly like the original: about 50% of the time, 23% from the levels rule and 35% from the band. We widened them to a 0.5 margin (1.8% false failure) and a 13–23 band (4%), about 6% combined. The cost is power: S1 can only rule out a drop of about 0.7 levels or more, or a large shift in acceptance.

First run: fail. The first run used c18d184, the version that passed every deterministic check.

Rule	Rewrite S1	Original enf10	Verdict
Levels per game run	0.533 (SD 0.571)	0.500 (SD 0.572)	pass: diff +0.033, Welch t 3.61, df 58, p 0.0003
Runs with an accepted model	26/30	18/30	fail: above 13–23
Median throughput	167.1 tok/s	165.85 tok/s	pass

The rewrite played as well on levels but wrote and passed a world model more often. The direction held against the pooled original runs, 58 of 90 accepted against 26 of 30 (Fisher p = 0.02; 0.04 against enf10 alone). No game was lower and 4 of 6 were higher. The comparison was not confounded by gate version: with our settings, the gate logic and default prompts in enf10 equal the rewrite's reference.

Cause. D2 had shown that every model input except prompt text was identical, so we compared model behavior and prompt content. The rewrite's model wrote `predict` earlier and more often: cells defining `predict` per run rose from 6.2 to 8.2, and the action count at the first `predict` fell from 6.8 to 5.2. Captures that passed the backtest rose from 21% (36 of 170) to 34% (80 of 233), and mean backtest accuracy from 0.466 to 0.587. The gain held at equal evidence levels, so the rewrite's models were not judged on easier data.

A code fingerprint tied this to the prompt. All 246 of the rewrite's `predict` cells parsed the board with the line-splitting idiom from the code example in the rewrite's prompt; none of the original's 187 did.

The two system prompts were the same length, 13,913 characters against 13,899, but their gate sections differed in content. The original's was 1,230 characters of interface and rules, with no code. The rewrite's was 2,048 characters and added a worked `predict` code template, a self-test snippet, and a construction strategy ("start with the one effect you are sure of"). It also spelled out acceptance details the original does not state. It dropped four facts the original states: the exact 8-action allowance, that the allowance is for learning what actions do, that an accepted model must keep predicting, and that an accepted model should be used to search for a plan. The rewrite's per-turn prompt also framed play as one-move experiments, where the original encouraged batching and search.

Some of the 8-run gap could be sampling noise. But the behavior shifts were large and systematic, and the template idiom appeared in 100% of `predict` cells.

Fixes. Three rounds of prompt changes followed, each re-checked deterministically:

- Round 11 (a909fbe): the gate section was cut to interface and rules, 1,238 characters against the original's 1,230, with the original's facts in the same order. The template, snippet and strategy were removed, and the turn framing and world-model instruction were restored.
- Round 12 (81a0446): a simulate-on-a-copy hint in the general Python guidance was removed. The verifier rated it the most likely remaining acceptance driver, because it asks for a state-transition function, which is functionally a `predict`. The original's emphasized search instruction, its edge-bar warning and its stop-probing advice were restored.
- Round 13 (52f5ac6): two orientation claims that make board prediction look easy were removed: that the visible board is the whole state, and that nothing moves between actions.

After each round, D2 still matched 1,504 of 1,504 eviction points with exact serialized sizes, and the 8-gram check found 0 prompt-text hits.

Second run: pass. The second run was registered before its data. It tested a new rewrite version, not a repeat of a valid run, under unchanged rules. It ran on a rented RunPod RTX PRO 6000 for $6.54.

Rule	Rewrite S1b	Original enf10	Verdict
Levels per game run	0.467 (SD 0.507)	0.500 (SD 0.572)	pass: diff −0.033, t 3.34, df 57.2, p 0.0007
Runs with an accepted model	19/30	18/30	pass
Median throughput	163.2 tok/s	165.85 tok/s	pass

Acceptance fell from 26 of 30 in S1 to 19 of 30. Two limits apply. The fixes were chosen after seeing S1 data, and S1b used the same 6 games, so S1b is not an independent confirmation. And rounds 11–13 changed several things together, so the result shows that the combined fixes closed the gap, not that the template alone caused it.

The fixes also changed what the model wrote. The template's line-splitting idiom, present in all 246 of S1's `predict` cells, appeared in 8% of S1b's 291, against 0% of the original's 187. Per game, runs with an accepted model were:

Game	Original enf10	Rewrite S1	Rewrite S1b
bp35	1/5	4/5	3/5
lp85	2/5	5/5	1/5
tn36	4/5	5/5	5/5
vc33	2/5	3/5	1/5
sb26	5/5	5/5	5/5
tu93	4/5	4/5	4/5
Total	18/30	26/30	19/30

In S1 no game was below the original. In S1b two games were below it (lp85, vc33), two above (bp35, tn36) and two equal, so the S1b total is near the original's without a uniform per-game match.

After S1b: one same-size reword. A later clean-room audit of the restored prompt items found one guidance sentence close to the original's wording ("write a scorer … or a search over candidate sequences"). We reworded it at the same length (123 characters), so the system message stayed at 14,065 serialized characters. The deterministic checks were re-run: 0 prompt-text hits in the 8-gram check and none in a 3-gram check on the new lines; record/replay matched in 30 of 30 runs on actions, token attribution, gate logs and eviction points, with 1,199 of 1,284 tool results identical and the rest timing, error wording, set order and memory addresses. S1b did not see the reworded prompt. Every later run did: the leaderboard submission (57752331, built from e2fa28e) and all nine arms of section 4 (all built from commits that contain the reword, 8958f5c). Arm A of section 4 is the unchanged harness on the reworded prompt (mean score 1.399, 16 of 30 game runs with a level, against the original's 1.41); it was not run as a non-inferiority test, so it shows the reword in use, not that it is equivalent.

The backtest_error check. One measure without a pass rule stood out: 27 backtest_error gate events, against 3 in the original and 6 in S1. We checked them before trusting the pass. All 27 are Python exceptions in the model's own top-level code when the gate replays a captured cell, 10 ZeroDivisionError and 17 NameError; none is a timeout or a harness error. The mechanism is the gate's capture rule, which is identical in both harnesses. A static check over every `predict` cell flags 3 of 187 (original), 6 of 246 (S1) and 28 of 291 (S1b), matching the observed counts. The shift is significant per cell ($p = 0.0004$) but not per run (8 of 30 runs against 3 of 30, $p = 0.18$). Each error rejects only one candidate, so it moved no S1 rule.

3.3 Lessons

1. Matching facts and sizes is not enough. At c18d184 the rewrite matched the original on every deterministic measure, including 1,504 of 1,504 eviction points and exact serialized sizes for every message template. Its prompts were new wording on the same topics. The live model still behaved differently: 26 of 30 runs accepted a world model, against 18 of 30. Deterministic replay holds the model's output fixed, so it cannot show how the model reacts to new text. In a clean-room rewrite of an LLM harness, prompt content is part of the behavior and needs its own test. The parity review that fixed it compared facts and emphasis element by element, not wording.

2. Templates change model output. The largest single difference was a worked code example in the gate instructions, and the model copied it: all 246 rewrite `predict` cells used its board-parsing idiom, against 0 of 187 in the original. With the template, first world models came earlier, and passing captures rose from 21% to 34%. A hint in the general guidance to simulate actions on a copy of the board asked for the same function under another name. Removing the template and the hint, and restoring the dropped facts, brought acceptance back to 19 of 30. An example in a prompt is an instruction; if the original has none, the rewrite should have none.

3. Check pass rules against noise before collecting data. The first draft of the S1 rules would have failed an identical rewrite about half the time. We found this by computing the false-failure rate under the null before running anything, and widened the rules to about 6%. The price was stated in the plan: S1 detects only large shifts. The same habit applied inside the deterministic checks. Running the original against itself set the noise floor: 15 D3 cells of set-order disagreement, and timing-digit noise in tool messages that moved no eviction point. Without that floor, set-order noise could not be told apart from a real difference.

4. Nine controlled tests of harness mechanisms on the rewrite

The verified rewrite was then used to test the gate and harness mechanisms directly. Every test was registered before its arm's data in `arc-offline-tests-prereg.md` (C200–C209). Statements marked [inferred] are interpretation, not measurement.

4.1 Setup

Harness. arc-wm-agent is a world-model agent for ARC-AGI-3 games. In each turn a language model writes Python cells that run in a sandbox; a cell can inspect the game, call `action(...)` to play one or more moves, and define a `predict` function that maps a board and an action to the next board. A world-model gate refuses actions once a level has used its probe budget (`PROBE_ACTIONS` = 8) unless the current `predict` has been accepted: it must be backtested on at least 3 informative moves (moves that change more than 2 cells) at 75% accuracy or better. The gate releases a blocked level after `PROBE_MAX_REFUSALS` = 8 refusals.

Model and server. All arms served the same model snapshot (`vrfai-qwen3-6-27b-fp8-hf-snapshot`, architecture `Qwen3_5ForConditionalGeneration`, max model length 65,536) with vLLM v0.19.0 on Kaggle, with `preserve_thinking` on, so past reasoning is re-sent in later requests. The GPU is an H100 **[inferred from the wheelhouse name `arc3-vllm-h100-wheelhouse-v3`; the run logs read for this draft do not name the device]**.

Games and budget. Six games from the public set (`PROBE_GAMES`: tn36, bp35, vc33, lp85, tu93, sb26), each played 5 times per arm, for 30 games per arm. Ten games run concurrently; each game has a cap of 2,700 s. The score of a game is `final_score` in `benchmark.json`. These settings match two earlier runs of the unmodified harness (s1 and s1b), checked against their `agent_env.json` and `run_config.json`.

The decoding-speed constraint. An audit of s1 and s1b, done before the mechanisms were written, found that decoding speed limits the score. A game generates about 45k tokens, about 1.08k per model

call, at about 16 tokens/s per stream, so one call takes about 65 s. The audit counted 41.8 model calls per game. In 15 of 30 (s1) and 13 of 30 (s1b) games the agent played 12 or fewer actions in the whole game. A turn ends when it plays an action or when a fixed 60 s turn budget (the "yield") runs out; the yield is checked only between requests, so it does not cut off a call in flight. Turns ended by the yield (444 in s1, 466 in s1b) outnumbered turns that played (411 and 354). From these counts the plan took model calls per executed action as the main lever and context size as secondary.

4.2 Mechanisms tested

The audit also traced specific faults in the code. The four mechanism groups below were written fresh for this harness from designs in earlier harnesses (sources are named in the plan).

M1. Earned-trust gate. - **M1a, capture on deny.** In the original harness, a refused `action()` raises inside the cell, the cell counts as errored, and a `predict` defined in that cell is not captured. Transcripts showed 8 of 65 refused cells defined `predict` in s1, 47 of 101 in s1b and 33 of 77 in a third run (paper-check). M1a captures and backtests `predict` from a refused cell, appends the verdict to the tool result, and corrects the forced prompt, which omitted the "3 informative moves" rule and said "board changed" where the gate needs more than 2 changed cells. - **M1b, deny evidence.** The refusal carries the last verdict in about 250 characters (accuracy, informative count against the minimum, first missed transition), re-injected into the next prompt while the level stays blocked. - **M1c, grant on probation.** A model confirmed on held-out moves on level L carries a grant into L+1: no refusals while it holds; two consecutive informative misses revoke it; it lasts one level. - **M1d, release valve.** A blocked level with no accepted model is released after 8 model calls, alongside the existing refusal cap (whichever fires first).

M2. No-progress breaker. - **M2a, no-op deny.** Refuses an action whose (board hash, action) pair already produced no change on the level. In the tested arm it ran in shadow mode: logged, never enforced. - **M2b, STUCK line.** A one-line fact in the next prompt when executed actions change nothing, cell code repeats, or refusals produce no capture. - **M2c, yield continuation.** After a turn that yielded without playing and whose history was kept, the next turn sends a short "turn still open" prompt with the live image (plus any deny or STUCK line) instead of the full turn prompt.

M3. One classifier with per-class gates. Each game is classified as keys, click or mixed from its valid actions; one `GateProfile` feeds the gate, the prompt and the logs. Enforced values stayed at the original constants for every class; alternative profiles were logged in shadow only. M3 therefore changes no behavior in these tests.

M4. Context economy and memory. - **M4a, monitor.** Per-request logging of timing, usage and cached tokens (logging only; present in all arms). - **M4b, image elision.** Board images are dropped from persisted history; only the live request carries one. - **M4c, call economy.** A prompt line states the remaining probe count and asks the model to batch probes; a batch longer than the remaining budget is truncated to it instead of refused whole. - **M4d, level memory.** On level completion the harness records the action count, the shortened action sequence and each label's effect rate, and shows them in later prompts as a "Solved levels" block.

Arms. One commit (4d134f5) built all three arms, selected by `WM_MECHANISMS`: - **A (`base`):** the original harness plus logging only (M4a, `game_class` and shadow fields on gate records). - **B (`m1a`):** A plus M1a. - **C (`all`):** B plus M1b-d, M2a (shadow), M2b, M2c, M4b, M4c and M4d.

Each arm's `session_start` record lists its active mechanisms, so the arm can be confirmed from its own logs. Before the runs, the test suite (340 tests) passed and a scripted self-test corpus fired every C path at least once.

4.3 Pre-registration and statistics

Each test was registered in `arc-offline-tests-prereg.md` before any data from its arm existed. C200 was registered on 2026-10-29; C201 and C202 on 2026-10-31.

Primary test (C200). C vs A only, on game score; a missing score counts as 0. The statistic is T = mean over C's 30 games minus mean over A's 30 games. The test is a one-sided Monte Carlo permutation test stratified by game: in each draw the 10 scores of each game (5 from A, 5 from C) are reshuffled between the arms; 100,000 draws, seed 20261029; p = (1 + #{T* >= T_obs}) / (1 + 100,000). Pass: p < 0.05. B is descriptive only.

Co-primary harm guard. The number of C's 30 games that clear at least one level must be at least the number in A (equal counts pass).

Ship rule. C ships only if the primary passes and the harm guard holds.

Power. The within-game SD over s1 and s1b was 1.2356 (60 games, df 54). At 30 vs 30 games, one-sided alpha 0.05 and power 0.8, the smallest detectable effect is +0.79 mean score (2.487 x 1.2356 x sqrt(2/30)). The simulated power of the stratified permutation test was 0.48 at +0.5, 0.83 at +0.8 and 0.97 at +1.0. The registration recorded as an accepted limit that gains under about +0.5 would likely read as no effect.

Failed runs. A kernel that ends without `benchmark.json` voids the arm, which is rerun once; a second failure means no ship. A game marked crashed inside a completed run keeps its recorded score.

Multiplicity for C201 and C202. Both follow-up arms are tested against the same arm A run. C201 keeps alpha = 0.05; C202 uses alpha = 0.025, so the registration bounds the combined false-ship risk at 0.075. At alpha 0.025 the smallest detectable effect rises to about +0.9. If both arms qualify, the one with the higher mean ships, and a tie goes to C2 as the simpler change. C200's own test against the same A is not part of that bound **[observation by the author of this draft, not stated in the registration]**.

4.4 Results of C200

All three kernels completed with `benchmark.json`; no score was missing and no game crashed (every game ended in state `gave_up`, the cap).

Game	A mean	B mean	C mean	C - A	Games with >= 1 level (A / B / C)
tn36	1.429	0.714	1.429	0.000	2 / 1 / 2
bp35	0.000	0.000	0.000	0.000	0 / 0 / 0
vc33	1.261	0.000	0.678	-0.583	2 / 0 / 3
lp85	2.222	2.778	1.111	-1.111	4 / 5 / 2
tu93	2.328	2.966	1.425	-0.903	5 / 4 / 4
sb26	1.156	1.766	1.639	+0.483	3 / 4 / 4
All 30	**1.399**	**1.371**	**1.047**	**-0.352**	**16 / 14 / 15**

Per-game means are over 5 plays each, computed from `benchmark.json`.

- **Primary:** T = -0.352, p = 0.8502. Fails.
- **Harm guard:** 15 < 16. Fails.
- **Verdict:** no ship for C. B's mean (1.371) was within 0.03 of A's; B is descriptive only.
- C was lower than A on three games (vc33, lp85, tu93), equal on two, and higher on one (sb26). With 5 plays per game, none of these per-game differences is tested.

Mechanism counts in C (from the prereg result): the release valve fired 13 times; the grant fired 3 times and was revoked 3 times.

4.5 Diagnosis: the continuation-prompt slowdown

This analysis was done after the C200 result was read. It is post hoc and descriptive, and it motivated the C201 arm; it is not a test.

Fewer calls per game. C made 34.8 model calls per game against A's 44.1. Since a call takes about a minute at this decoding speed, fewer calls means fewer chances to act in the 2,700 s cap.

Continuation calls were long. C's yield-continuation turns (372 of its 1,043 calls) generated 1,681 tokens and took 103 s per call on average. Its full-prompt turns generated 1,108 tokens and took 65 s, against A's 1,088 and 63 s.

Like-for-like comparison by turn type. Splitting calls by their position relative to yields:

Calls	A	B	C
Right after a 60 s yield	1,264	1,206	1,681
Fresh turns	846	817	1,108
Before any yield in the game	407	376	429

(Mean generated tokens per call.) Before any yield, the three arms generated similar amounts. After yields began, C's calls were longer both right after a yield (where C sends the continuation prompt and A and B send the full prompt) and on fresh turns. A's full-prompt figure of 1,088 tokens includes its post-yield calls, which also receive the full prompt; the fresh-turn figure of 846 excludes them **[inferred from the two definitions; the source does not state the split]**.

The excess grew over the game. C's fresh-turn excess over A grew with the number of yields so far in the game: +21% after 1-3 yields and +59% after 4-8. C's prompts also grew 0.8k-3.5k tokens above A's despite image elision (M4b).

Interpretation [inferred]. The long continuation turns appear to lengthen later turns through the kept history: the model's own long outputs stay in context and later outputs grow with them. This is a hypothesis drawn from the same data it explains. C bundles nine changes, and image elision (M4b) or another C-only change could contribute. C201 tests the continuation hypothesis on new data. **C201 refuted it** (section 4.7.1): C2, without the continuation prompt, generated even more per call.

4.6 The feedback gap

A second finding from the C200 logs concerns what the model is told about its own world model. In arms A-C, 66-126 failed `predict` captures per arm came from cells that were not refused, and the model was never shown the verdict. A gate note attached to a cell that ran without an error was also dropped by `format_tool_result`. M1a fixed the refused-cell path only: the verdict reached the model after a refused cell, but a failed backtest after an ordinary cell stayed silent in every arm.

[inferred] This means the gate's main information channel to the model, the backtest verdict, was missing in the case where the model was most likely to be iterating on `predict` without being blocked. The per-arm breakdown of the 66-126 range is not in the sources used for this draft.

C202 (arm D) closes this gap.

4.7 Follow-up tests C201-C204

Each arm runs on the C200 common settings and is compared with C200's arm A run, reused as is.

4.7.1 C201: arm C2 (`all-nocont`)

- **Change:** `all` with the yield-continuation prompt (M2c) off. After every yield the full turn prompt is sent, as in A. Every other C mechanism is unchanged. Commit 2e9893d; label `wm-pc-c2`.
- **Why a new arm:** the hypothesis came from C's data (section 4.5), so C2 is tested on new data; C's run is not reused.
- **Rule:** the C200 primary test and harm guard with C2 in place of C: stratified one-sided permutation test, 100,000 draws, seed 20261029, pass at $p < 0.05$; C2's games with at least one level must be at least A's 16. Ship C2 if both hold; otherwise no ship. Smallest detectable effect +0.79.
- **Registered mechanism predictions (descriptive, not part of the ship rule):**
  - If continuation was the whole cause, C2's post-yield and fresh-turn generation return to within about 10% of A's (about 1,260 and about 850 tokens per call), and C2 makes about as many calls per game as A (44).
  - If C2's fresh-turn generation stays well above A's (at least 1,000 tokens per call), image elision (M4b) or another C-only change is the next suspect.

- **Reading:** `scripts/read_arm.py` (analysis only), which applies the registered rule and stops first on a missing `benchmark.json`, a wrong game count, a manifest naming another arm, a missing score or a crashed game. Its shuffle order differs from the ad hoc C200 script: on C200's C vs A it reads p = 0.8514 against the recorded 0.8502, with the same verdict.
- **Results (read 2026-10-31):** NO SHIP. Mean score C2 0.829 vs A 1.399 (T = −0.570, p = 0.959); games with at least one level C2 11 vs A 16, so the harm guard also failed. No game crashed. The second prediction held: C2's post-yield calls generated 1,878 tokens (A 1,265, C 1,681), its fresh-turn calls 1,166 (A 847, C 1,108), and it made 32.5 calls per game (A 44.1). The continuation prompt was not the cause of C's slowdown.
- **Diagnosis after C201 (descriptive, post hoc, from the request logs and prompt snapshots of A, B, C and C2):**
    - The excess is on the first call of each turn (C2 1,555 vs A 1,089 tokens); later calls within a turn match A (1,116 vs 1,079).
    - It is not tied to the extra prompt lines: C2 first calls whose user message carried no gate, probe or memory line averaged 1,894 tokens vs A's 1,144.
    - It appears only once the history holds earlier turns: with fewer than 7 messages the arms match (297 vs 299; 472 vs 471 tokens).
    - At matched history size (13-30 messages), A's history held 7 board images and 13.5k characters of kept reasoning; C2's held 1 image (M4b elision) and 19.0k characters, and its next reasoning was 1,668 vs 1,160 tokens. B (no elision) matched A throughout.
    - **[inferred]** Elision appears to start a loop in which the model writes more reasoning in place of the missing past boards, the reasoning is kept in context, and later turns lengthen. Elision and the other C-only mechanisms always fire together in C and C2, so this is not shown causally.

4.7.2 C202: arm D (`all-nocont-fb`)

- **Change:** C2 plus feedback after every cell that defines `predict`, not only after a refused cell: the gate's verdict in the tool result; a failed backtest's error line and the rule that captured definitions cannot use names from other cells; a wrong board shape named in the first-miss sentence. Commit 1ff1c54; label `wm-pc-d`. Logging changes on this commit (a row for failed requests, a shape column on backtest rows) apply to all arms and do not change what the model sees outside `all-nocont-fb`.
- **Rule:** as C200 with D in place of C, but pass at p < 0.025; harm guard D's games with at least one level at least A's 16. Smallest detectable effect about +0.9. If C2 and D both qualify, the higher mean ships; a tie goes to C2.
- **Reported (descriptive):** how often the new verdict note fired and whether the next `predict` changed after it, plus C201's descriptive items.
- **Results (read 2026-10-31):** NO SHIP. Mean score D 1.153 vs A 1.399 (T = −0.247, p = 0.769); games with at least one level D 15 vs A 16. The highest mean of the `all` family (C 1.047, C2 0.829). The whole positive difference sits in one game: sb26 scored 2.78 in four of five plays (A: two scoreless plays), mean 2.72 vs 1.16; every other game was flat or below A. The token excess persisted (post-yield 1,614, fresh-turn 1,124 tokens per call; 36.2 calls per game).

4.7.3 C203: arm E (`all-nocont-fbx`), stopped before data

Arm E added four fixes from the C200 logs to D (a rejected action no longer costs a probe; a gate release is announced; traceback line numbers count from the model's own cell; capture keeps only self-contained constants). After C2 and D failed with the same token excess, which E would also carry, the run was cancelled after about 32 minutes and its output was never read. Its alpha was not spent, so the combined false-ship risk stays at most 0.10 (C201 0.05, C202 0.025, C204 0.025).

4.7.4 C204: arm F (`all-nocont-fbx-img`)

- **Change:** E with image elision off: kept user messages keep their board image, as in A and B. Commit 9a1c985 (which also carries logging-only changes: request failures in a row are capped and printed, every game records an end reason, crash tracebacks are printed, the run's settings are written next to its results). Label `wm-pc-f`.
- **Rule:** as C202, pass at $p < 0.025$; harm guard F's games with at least one level at least A's 16. Before any submission, the per-game breakdown and a leave-one-game-out rerun of the test are reported.
- **Registered predictions:** if elision was the cause, F's fresh-turn and post-yield generation fall to within about 10% of A's and F makes about 44 calls per game; if F stays at 1,000 or more tokens per fresh turn, another C-only mechanism is responsible.
- **Known confound, registered before F data:** the harness's token estimator counts a kept image's base64 text at characters/3, about 479 estimated tokens per board image, where the model's real cost is far lower (about 64-66 tokens by the vision encoder's patch arithmetic **[inferred, not measured]**; estimated/real prompt tokens 1.28 in A vs 1.18-1.19 in C2 and D). Kept images therefore use up the history budget sooner and older text turns are trimmed earlier. F restores A's regime on both counts, so it cannot separate the images from the shorter text history.
- **Results (read 2026-10-31):** NO SHIP under C204 (mean F 1.565 vs A 1.399, $T = +0.166$, $p = 0.330$ against the 0.025 threshold); harm guard passed (19 vs 16 games with at least one level). The first arm of the series above A. No game crashed; all 30 ended at the time cap.
- **Registered prediction held:** fresh-turn generation 854 tokens per call (A 847), post-yield 1,324 (A 1,265), 45.5 calls per game (A 44.1). With past board images kept, the token excess of C, C2 and D disappeared, which identifies elision as its cause, within the registered confound (kept images also shorten the kept text history through the token estimate).
- **Where the gain came from:** per-game mean differences tn36 +0.71, sb26 +0.69, tu93 +0.41, lp85 +0.13, bp35 +0.10 (its first score in any arm), vc33 −1.04. Dropping any one game leaves the mean difference positive (+0.06 to +0.41).
- **Mechanism telemetry (descriptive):** captures with a failed backtest fell from D's 20 backtest-error rows to 5, and NameErrors in transcripts from 83 to 46 (the self-contained capture fix); accepted predicts 66 (D 34, A 82); level changes 18 (A 15); the release notice appeared in the prompts of released levels. vc33 lost score through efficiency, not clears: F cleared its first level in 2 of 5 plays as A did, but with 54 and 13 actions against A's 8 and 7 (scores 0.06 and 1.04 against 2.73 and 3.57) **[inferred: released levels let the model click freely, and the per-level score penalizes extra actions]**.

- **Draw decision:** F met the draw rule registered before its data (mean at least A's, harm guard, gain not carried by one game). That rule accepts an arm that is no worse than A on 30 games without establishing a gain; a draw submission made under it is reported as such.

4.7b Gate-config family: C205 (arm G), C206 (arm H), C207 (arm F2), C208 (arm I), C209 (arm I2)

Registered as a separate family (user decision), each test at alpha = 0.05 against the same arm A run.

- **C205, arm G: the base harness with the gate's probe budget raised from 8 to 16.** Motivation (descriptive, from A-D's logs): 48 of 150 runs never cleared a level and stopped at 10 or fewer level-1 actions; all 10 failed lp85 runs stopped at exactly 8; 210 of 237 gate refusals would not have fired at 16. **Result:** NO SHIP (mean G 1.524 vs A 1.399, T = +0.124, p = 0.362; games with at least one level 16 vs 16). lp85 and sb26 cleared level 1 in all 10 G runs (A: 4/5 and 3/5), and lp85's level 2 reached 13-26 actions where A stopped at 8. The gain rests on sb26 (64% of the positive total; without sb26 the mean difference is -0.154), and G lost on tn36 (-0.71) and vc33 (-0.91).
- **C206, arm H: F plus the 16-probe budget.** Registered after F's result. **Result:** NO SHIP (mean H 1.586 vs A 1.399, T = +0.187, p = 0.306; games with at least one level 21 vs 16). lp85 and sb26 cleared level 1 in all 10 runs, as in G. The gain rests on sb26 (69% of the positive total; without it the difference is −0.089). tu93 fell by 1.14, mostly from level-2 clears (0 of 5, against 1 in A and 2 in F and G) and partly from efficiency (level 1 in 24-30 actions against A's 20-22).
- **C207, arm F2: an exact replicate of F,** registered because F was the only arm that met the draw rule and several arms had been checked. **Result:** F2 1.302 (below A); pooled F + F2 1.434 vs A 1.399 (T = +0.034, p = 0.458). F's draw qualification was withdrawn under the condition registered before F2's data. F's 1.565 was run-to-run noise; across 60 games F and A are indistinguishable, although F and F2 both kept A's call economy.
- **C208, arm I: H plus speculative decoding through the model's multi-token-prediction (MTP) head.** Every behavioral arm stayed within noise of A inside a fixed budget of about 45k generated tokens per game, and the GPU was saturated at 8-10 concurrent games (total generation 207 tok/s at 8, 168 at 10), so only faster decoding could add model calls. The FP8 checkpoint declared one MTP layer but shipped no MTP weights; the 15 BF16 `mtp.*` tensors of the full-precision model were grafted on and kept out of the FP8 scheme. A smoke run measured a mean acceptance length of 2.71 tokens per step on game traffic (2 draft tokens; at most 3) and about 38 tok/s per stream at 6 concurrent games against about 26-27 without MTP **[the 6-stream baseline has 2 samples]**. **Result:** SHIP under C208 (mean I 2.222 vs A 1.399, T = +0.823, p = 0.0249; 24 vs 16 games with a level), with 57.8 model calls per game against A's 44.1. It was the fourth test in its family; p = 0.0249 also clears a family-wise bound of 0.10/4 = 0.025, narrowly.
- **C209, arm I2: an exact replicate of I. Result:** I2 1.523; pooled I + I2 1.872 vs A 1.399 (T = +0.473, p = 0.076), so C208 is reported as **not replicated** under the criterion registered before I2's data. MTP's speed effect did replicate (55.2 calls per game; acceptance about 2.7). Its score effect did not reach significance on 60 games, although every game except vc33 was non-negative

in the pool. I versus H, the same settings without MTP, was +0.64 (p = 0.078) and I2 versus H was −0.06.

Why the budget binds. Decisions per game ≈ generation speed × cap ÷ tokens per call (about 17 × 2,700 ÷ 1,050 ≈ 44 in arm A). With the GPU saturated, generation speed is fixed, so a harness change can only alter tokens per call: gate rules, notes and feedback move calls between kinds of reasoning without adding any, and image elision removed calls by lengthening reasoning. Speculative decoding is the only tested change that raises the numerator. [inferred]

What the series shows [inferred from all nine arms]. At this decoding speed the harness's score is bounded by the number of model calls a game gets in 2,700 s. Changes to what the model is told or allowed (gate rules, feedback, prompt lines, images) moved the 30-game mean by less than run-to-run noise, and one of them (image elision) cost about 25% of the calls through longer reasoning. The only change that added calls, MTP, raised calls per game by 25-30% in both runs and the pooled mean by +0.47, which 60 games cannot separate from noise.

Draw rule and its false-pass rate

At 30 games the registered tests need about +0.8 (alpha 0.05) to +0.9 (alpha 0.025), while the levers found in the logs were estimated at +0.15 to +0.35. For the competition's daily draws the user registered, before F's and G's data, a separate rule: an arm qualifies if its mean is at least A's, its harm guard holds, and the mean difference stays non-negative with any one game removed. A simulation that draws null arms from A's own per-game scores (5 plays per game, 20,000 draws) puts the chance that an arm no better than A meets this rule at 25.9%, and the chance that at least one of three such arms meets it at 59.3% **[a lower bound: resampling A ignores day-to-day drift]**. A qualifying arm is therefore evidence of "not worse than A on these games", not of a gain; C207's replicate condition was added for that reason.

4.8 Limitations

- **Sample size.** Thirty games per arm (6 games x 5 plays) give a smallest detectable effect of +0.79 mean score at alpha 0.05 (about +0.9 at 0.025). Gains under about +0.5 would likely read as no effect, a limit registered before the runs. A null result here does not show that any single mechanism is ineffective.
- **Bundled arm.** Arm C changes nine things at once, in both directions: M1 lifts refusals while M2a would add them (M2a ran in shadow, so in practice it did not). C200 cannot attribute C's deficit to one mechanism; section 4.5 names a suspect, and C201 tests it.
- **Run-to-run variance.** Earlier runs of the same unmodified code on the same games and settings scored 1.12 (s1) and 1.16 (s1b); the s1/s1b audit also recorded one game (lp85) going from 5/5 to 1/5 between runs. Arm A, also the unmodified harness, scored 1.399 in this test. The gap between A and the earlier runs is about as large as the gap between A and C, so a single run per arm leaves wide uncertainty about each arm's level.
- **Timing within C200.** A and B started about five minutes apart and ran concurrently; C started about three hours after A on the same day (`start_time` in `benchmark.json`: 00:29, 00:34 and 03:25 UTC, Oct 31). The arms did not share a time window.

- **Cross-day comparison arm.** C201 and C202 reuse C200's arm A run from Oct 30 instead of running a fresh A alongside them. Any day-to-day change in the environment (Kaggle hardware, server state) is confounded with the arm difference. The registrations record this as a known departure.
- **Post hoc diagnosis.** Section 4.5 was written after the C200 result and from the same data; C201 then refuted its continuation explanation. The elision explanation in section 4.7.1 is again post hoc, from C, C2 and D, and C204 tests it with a registered confound (the image token estimate also shortens the kept text history).
- **Stopped arm.** C203 (arm E) was cancelled before its data was read, by a decision made after C201 and C202 were known. Its fixes were not tested on their own.
- **Game set.** Six public games, chosen for earlier harness work (the `PROBE_GAMES` list). The results may not hold on other games.
- **Unreported secondaries.** The C200 registration listed further descriptive measures (games with 12 or fewer actions, calls per executed action, cached tokens per request, and counts for captures after a deny, deny follow-ups, shadow no-op outcomes, STUCK lines and `memory_use`). Only the valve and grant counts are in the sources used for this draft.

Attribution, licenses and availability

- **Original harness.** The rewrite re-implements the behavior of The Duck by Tufa Labs (`Tufalabs/duck-harness` on GitHub, the winning solution to ARC-AGI-3 Milestone 1, with a technical write-up in the competition's Kaggle discussion, topic 717133). It was written from a behavioral spec by agents without access to that source. As of 2026-11-01 the Duck repository has no LICENSE file and GitHub reports no license; its `ARC3-Inference` package declares an MIT license classifier.
- **Model.** Qwen/Qwen3.6-27B and the vrfai/Qwen3.6-27B-FP8 snapshot are Apache-2.0. The MTP draft head of section 4.7b is the 15 BF16 `mtp.*` tensors of Qwen/Qwen3.6-27B.
- **Code.** https://github.com/quanticsoul4772/arc-wm-agent, under CC0 1.0: the rewrite, the gate and its mechanism arms, the MTP setting and the analysis scripts (`read_arm.py`, `robustness_check.py`). The verification repository with recorded replays and corpora is not released, because those may contain third-party prompt text.